



University of Glasgow | School of
Computing Science

WebGraph: A Bookmark Management and Visualisation Plugin for Google Chrome

Raphael Taylor - 2940447T

School of Computing Science

Sir Alwyn Williams Building

University of Glasgow

G12 8RZ

A dissertation presented in part fulfilment of the requirements
of the Degree of Master of Science at the University of Glasgow

30/08/2024

Abstract

This report details the development of the browser extension “WebGraph”. This application intends to add bookmark management and visualisation tools to chromium-based browsers. Through my research, I found that existing tools often fall short, offering very limited functionality of this kind. WebGraph provides an interactive, graph-based visualisation of saved bookmarks that is intended to integrate seamlessly into the browser and improve the organisation of users saved content.

The software architecture comprises Google’s Manifest V3 API, which works closely with WebGraph to store saved websites and their metadata in the browser’s local storage, using a hybrid of Chrome.storage API and IndexedDB for larger Base64 images. For the frontend, the D3.js visualisation library is used to create fully interactive and customisable network graphs of the user’s saved data. The d3-force module was used to create custom forces that were made specifically for this application.

WebGraph was evaluated by 8 participants online, and 3 participants in person, and it received an average score of 4.34 from its online evaluation. The testing revealed that users generally enjoyed using the application, and that mapping their knowledge on an interactive 2D plane did in fact accomplish the objectives set out at the beginning. Testing showed that the application is successful in representing semantic relationships between websites and user-defined categories/tags. It also showed that WebGraph improved information retrieval and allowed users to find their saved content more easily. And finally, it showed that users found the extension to integrate seamlessly with the browser, which was a central objective of the project. Future work involves adding a tree-based web history visualisation to further extend WebGraph’s capabilities.

WebGraph can be downloaded for free from the Chrome Web Store. Simply search for “WebGraph” at URL: `chromewebstore.google.com`

Contribution Statement

My project is a Google Chrome extension that I coded from scratch. GitHub Copilot was used to troubleshoot bugs in the code and for testing purposes. Generative AI was also used for boilerplate code. A minified version of the JavaScript library D3.js is the only code in the repository that was not written by me. All classes and methods included in the UML were designed and written by me.

Acknowledgements

I want to thank my supervisor, Dr Yutian Tang, for his much-valued support during this project.

Contents

Chapter 1	Introduction	1
1.1	Aims and Objectives	1
Chapter 2	Analysis & Requirements	2
2.1	Existing Visualisation Tools	2
2.2	Inspiration	3
2.3	User Personas	5
2.4	Requirements	6
Chapter 3	Design and Implementation	7
3.1	User Interface	7
3.2	Data Structure and Database	8
3.2.1	Chrome.storage API and IndexedDB.....	8
3.3	Graph Visualisation	9
3.3.1	D3.js and WebCoLa.....	10
3.4	UML and Sequence Diagram	11
3.5	Deployment to Chrome Store	14
Chapter 4	Testing and Evaluation	16
4.1	Performance and Optimisation	16
4.2	Online User Evaluation	18
4.3	In Person Evaluation	20
Chapter 5	Conclusion	21
5.1	Achievements	21
5.2	Limitations	21
5.3	Future Work	22
Chapter 7	References	23
Appendix A	Ethics Checklist	1

Chapter 1 Introduction

The internet has become an indispensable part of modern life, with web browsing being a central activity for many individuals. However, the organisation, categorisation, and navigation of web data present significant challenges for users because of inadequate software design [1]. In this context, browsing data encompasses browser tabs, bookmarks, and browsing history, all of which currently lack effective in-browser organisation and visualisation tools. A major decision within this project was deciding which of these types of browsing data I want to attempt to visualise to produce a beneficial software product that is an appropriate scope for 12 weeks of project time.

The existing methods for storing and visualising web browsing data are inadequate and present an opportunity for improvement. A more sophisticated system could potentially serve multiple purposes, functioning as a reference manager, a to-do list, a bookmark manager, and more. This multifaceted approach could enhance the user's ability to manage and understand their browsing data more effectively.

1.1 Aims and Objectives

The primary aim of this project is to develop a Google Chrome extension that addresses the limitations of current the current tools, both vanilla and third-party. This plugin aims to visualise web bookmarks in an interactive map using graph data structures to represent the semantic relationships between visited websites and user-defined categories or tags. It also aims to improve information retrieval to let users quickly find and access saved content.

A key objective is to develop a system that will enable users to discover patterns and connections in their web usage. The application should help people map their knowledge whether that's academic, work related, or just for pleasure.

The project also aims to create a user-friendly interface that seamlessly integrates with the Chrome browsing experience, from installation to more advanced features. By achieving these objectives, this project will provide a more organised, intuitive, and visually appealing approach to managing browsing data.

Chapter 2 Analysis & Requirements

The development of my app is driven by several key inspirations that stem from personal experiences with software like Obsidian and Zotero, but also a broader frustration with the limitations of current web browsers and their third-party plugins. This chapter will investigate these ideas, highlighting the shortcomings in modern browsers and the existing tools available for them. By examining these areas, the need for an improved solution becomes evident, laying the groundwork for WebGraph's design.

2.1 Existing Visualisation Tools

Vanilla Chrome

The vanilla Chrome bookmark manager uses a basic tree structure like a file directory system, lacking any truly interactive visualisation features. As the number of bookmarks increases, the bookmark bar becomes cluttered and difficult to navigate. It does not support the creation of complex semantic relationships between bookmarks beyond simple folder categorisation. Additionally, the search functionality is rudimentary, offering no advanced filtering or sorting options. This is a well-known limitation, leading to other developers to create alternative solutions.

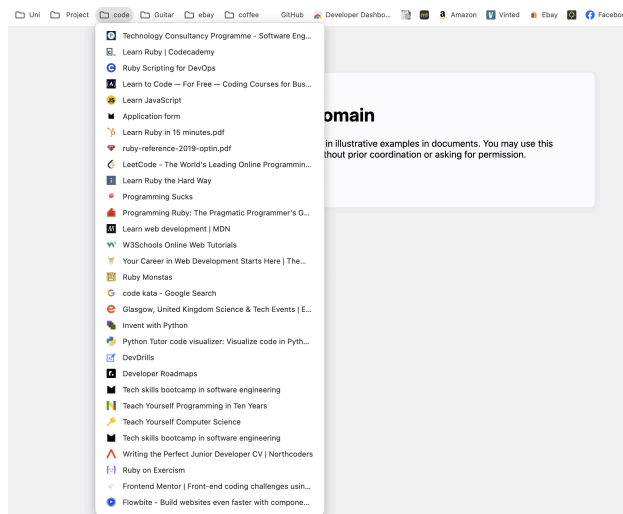


Figure 2.1: Screenshot of vanilla Chrome's bookmark bar

Third Party Plugins

There are several Chrome plugins already on the Web Store that attempt to visualise browsing data. Some of the most popular of these are Tab Manger Plus [2] and Simple Tabs. They have a combined user count of over 300,000 on Chrome. Tab Manger Plus has a intuitive user interface that uses the frontend framework React to create draggable tabs that can be placed in different groups. This extension is clearly helping many users fight the problem of tab overload,

but there are still problems with these extensions. In Tab Manger Plus, folders are separated by different windows of Chrome, which can easily lead to a cluttered desktop. Further, the user is the one who must organise structure themselves. There is no automatic or algorithmic creation of semantic structure.

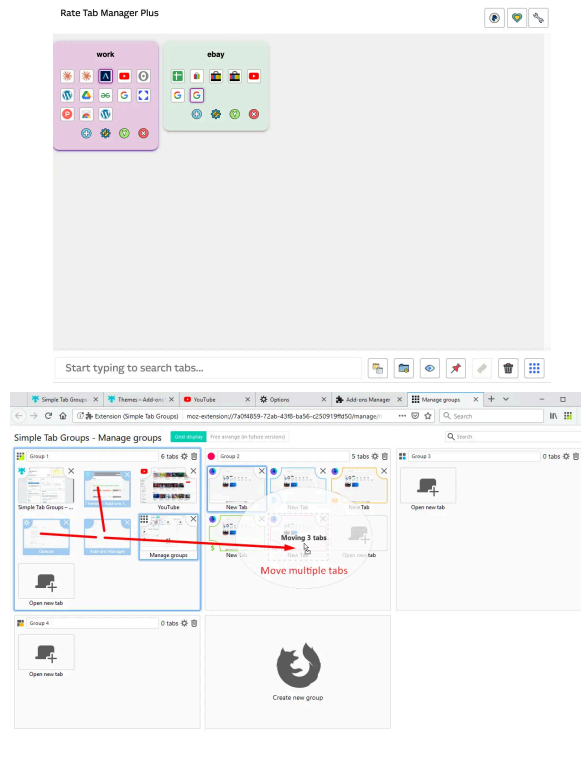


Figure 2.2: Screenshots of Tab Manager Plus’s UI and of Simple Tabs group’s UI

2.2 Inspiration

Obsidian

Throughout my degree, I extensively used a programme called Obsidian. This application leverages the Zettelkasten method [3], which is a note-taking technique that creates a personal library of interconnected Markdown notes. These notes can link to and backlink from each other, forming a dynamic network of notes and links, similar to Wikipedia. One of Obsidian's standout features is its graph view, which allows users to visualise a network graph of their notes. This visualisation proves immensely useful, as it reveals the semantic structure of the notes and facilitates activities such as organisation, pattern recognition, and the discovery of relationships between ideas that might otherwise go unnoticed.

Internet Maps

In addition to Obsidian, another major inspiration for this application was the various attempts over the years to map the vast expanse of the internet [4]. There are numerous methods to create visualisations of the web, each with its own strengths and limitations. However, a recurring thought during these explorations was the potential usefulness of a tool that could generate a real-time

map of one's own web activity. Such a tool could offer a more personalised and immediate understanding of how one navigates the web, revealing patterns, preferences, and connections that standard browser tools fail to capture.

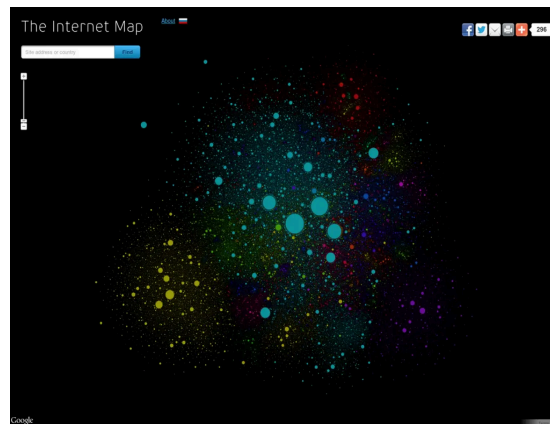
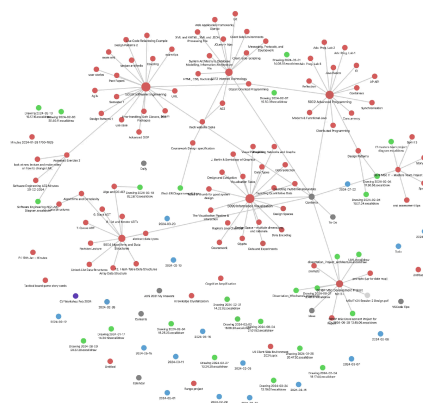


Figure 2.3: The Internet Map

Figure 2.4: My personal Obsidian graph view with study notes



Zotero

Zotero, a reference management software that utilises a multiple tagging system for organising references, played a crucial role in informing my design decisions for the app. Although the concept of tagging is not new, Zotero's implementation encouraged me to explore alternative ways to categorise web activity. In past dissertations for this course, webpages were the by far the most cited resource type, which is unsurprising given the internet's central role in information sharing, even in academic contexts. Zotero significantly influenced my decision to base the extension on a multiple tagging organisational structure. This began my conceptualisation of a different kind of data structure than traditional hierarchies for organising a user's internet browsing data.

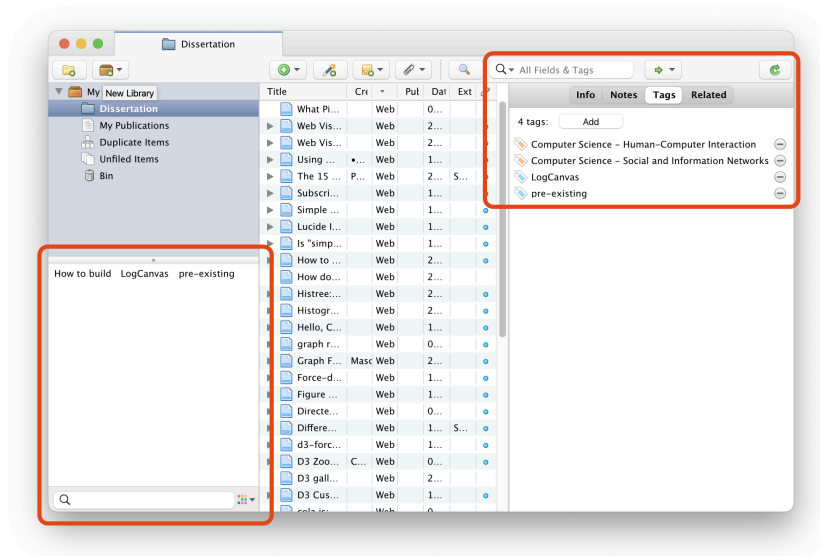


Figure 2.5: Screenshot of Zotero reference manager showing its multiple tagging file system

File Systems

The organisation of digital information has relied on hierarchical file systems for a long time, despite their limitations. As noted in the paper "Not Dead Yet: Hierarchical File Systems Won't Die" [5], there is widespread acknowledgement that most users do not effectively utilise hierarchical namespaces. Yet these structures persist as the default for both storage and user interfaces across most computing systems. In the context of modern web browsing, the typical browser interface presents open tabs in a linear sequence, while bookmarks are often organised into folder-like structures, mirroring traditional file systems.

The authors of the paper offer graph file systems as a more flexible structure by allowing files to be interconnected through multiple paths. Graph file systems enable more intuitive navigation, as users can traverse through data in a manner that reflects the natural, non-linear way they think about information. For instance, in a web browsing context, a single webpage could be associated with multiple tags or topics, and users could navigate through these associations without the constraints of a rigid folder hierarchy.

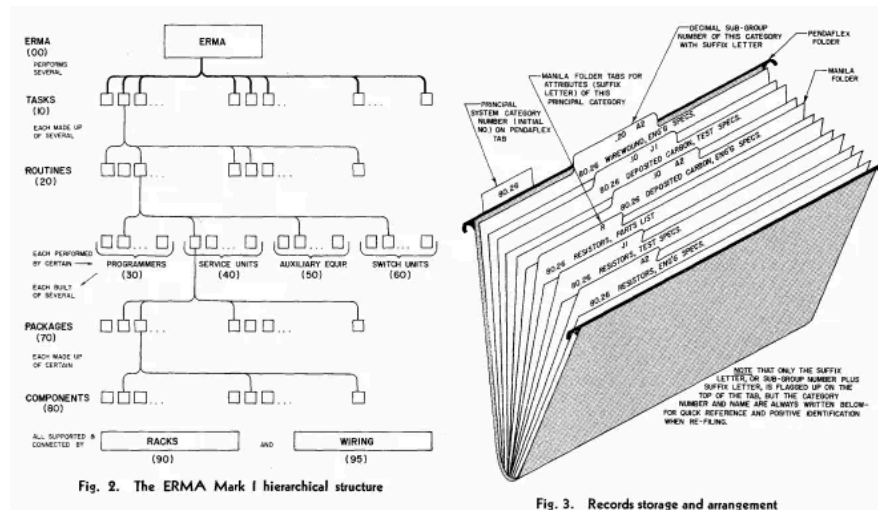


Figure 2.6: ERMA Diagram mapping file folders to hierarchy. Possibly the earliest record of hierarchical file structure. [6]

2.3 User Personas

I envisage three main personas who could benefit from using the WebGraph extension. The first is the Information User, who seeks to categorise academic research across different topics and visualise connections between links. This user also wants to see semantic structure between groups of webpages, aiding in the organisation and understanding of complex information landscapes. They would need an effective way to search for content as this user's saved data could become vast.

The second is the Task-Oriented User, who could benefit from seeing their graph as a tree that needs pruning, representing tasks being completed. Instead of experiencing tab overload, their tasks are stored visually in one place, preventing them from feeling overwhelmed and allowing for more efficient task management.

The third persona is the Leisure User, who could use the extension to organise and explore personal interests. They appreciate the organisation and navigation of saved links but, unlike the Task-Oriented User, are not attempting to prune their graph. Instead, they use the visualisation to explore and discover connections within their areas of interest.

2.4 Requirements

MoSCoW	Requirement
Must Have	Users must be able to save website data including URL and title to a local database
Must Have	Users must be able to add a tag/multiple tags to saved websites
Must Have	Users must be able to view an interactive force-directed graph of their bookmarks and tags
Must Have	Users must be able to search for nodes within the graph
Must Have	Users must be able to view detailed information about saved sites in a sidebar

Must Have	Users must be able to customise basic graph visualisation parameters
Must Have	Users must be able to store all data locally in the browser
Should Have	Users should have a way to edit tags from the graph view
Should Have	Users should have a way to remove bookmarks and tags
Should Have	Users should be able to capture a snapshot of the site they want to save
Should Have	Users should be able to save notes for any site that they save
Should Have	Users should be able to import and export graph configurations as JSON files
Should Have	Users should be able to switch between different "spaces" or graph configurations
Should Have	Users should be able to use a "local view" feature to focus on subsets of the graph
Should Have	Users should be able to drag and reposition nodes within the graph
Should Have	Users should be able to see website favicons for corresponding site nodes
Could Have	Users should be guided through the concept of the plugin when installed for the first time
Could Have	Users could be able to apply different colour schemes to nodes and edges
Could Have	Users could be able to view analytics about their browsing and bookmarking habits
Could Have	Users could be able to share their graph configurations with other users

Table 2.4: List of requirements

Chapter 3 Design and Implementation

3.1 User Interface

The UI is divided into three main sections: a settings panel on the left, a central graph visualisation area, and a details sidebar on the right. This layout follows Shneiderman's Visual Information-Seeking Mantra of "Overview first, zoom and filter, then details-on-demand." [7]. Whenever the page is refreshed, the graph automatically shows the entire network, to give the user an “overview” of their bookmarks. In the central graph area, users can interact with the graph through zooming, panning, and clicking on nodes, adhering to the "zoom and filter" part of the mantra.

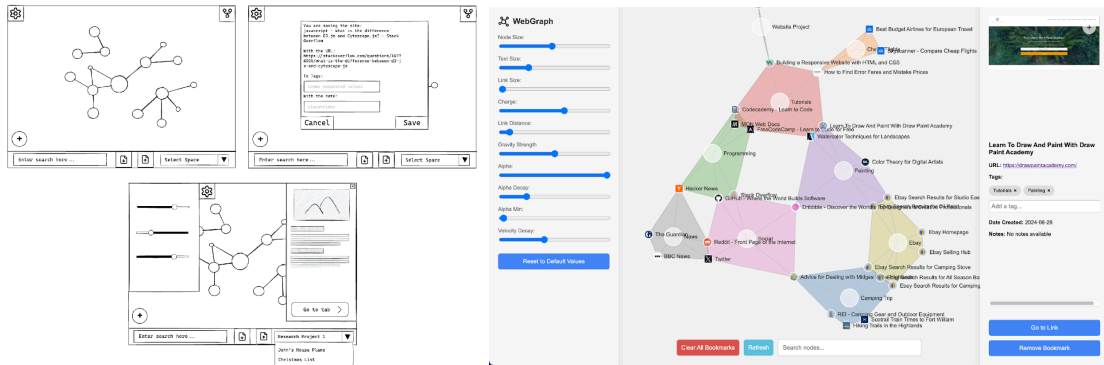


Figure 3.1: Initial wireframes and final product

The right sidebar displays “details-on-demand”, fulfilling the last part of Shneiderman's mantra. When a user clicks a node, it updates with specific information about the selected website, including tags, title, URL, creation date, and custom notes. I also implemented a snapshot preview that expands on hover, giving users a quick visual reference without leaving the main interface.

I initially made backend adjustments to the d3-force [8] physics settings to control the physical attributes of the graph, like charge, collision strength, etc. To allow for real-time updates and immediate visual feedback, I incorporated sliders into the HTML as a left-hand sidebar, enabling experimentation during testing to fine-tune these physics settings. I decided to retain some of the most engaging physics attributes, offering them as interactive elements for users to customise their graph experience. This allows for a more personalised interaction with the graph's dynamics.

When comparing the initial wireframes to the final product, a notable change is the addition of visual tag groupings or constraints. D3.js has a very useful method called `d3.polygonHull` that returns the convex hull of an array of points using Andrew's monotone chain algorithm [9]. The result is a subset of the original array, arranged in clockwise order. This became the method used to create coloured grouping backgrounds for every tag. The colour is automatically generated using D3's built-in ordinal data colour palettes. The decision to remove the JSON switching, importing, and exporting features were down to scope. They were not crucial to the MVP, so they were put aside for now.

3.2 Data Structure and Database

As I continued to research graph and set theory [10], I became familiar with the terminology of the data structure that I was getting closer to realising. I knew conceptually the rules I had in place for the data structure and how this could be visualised by a graph, but learning some set notation helped with solidifying the rules of the data structure.

Let S be the set of all sites and T be the set of all tags. These sets are disjoint, meaning they have no elements in common: This ensures that no entity in the data structure is both a site and a tag.

$$S \cap T = \emptyset$$

The relationship between sites and tags can be defined as a set R of ordered pairs where each pair represents a connection between a site and a tag.

$$R \subseteq S \times T$$

This notation captures the many-to-many relationships, where a site can be linked to multiple tags and vice versa. Therefore, the data structure in WebGraph can be represented as a bipartite graph – that is, a graph whose vertices can be divided into two disjoint sets such that every edge connects a vertex in one set to a vertex in the other set. [11]

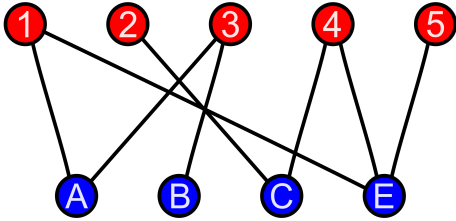


Figure 3.2: Bipartite Graph



Figure 3.3: Bipartite graph as seen in WebGraph

ite Graph

3.2.1 Chrome.storage API and IndexedDB

I faced an important decision regarding how to store the various types of data the extension needed to manage. WebGraph required storage for a range of data types, including bookmarks, tags, and potentially large image snapshots of websites. This need led me to consider two main options: the Chrome.storage API and IndexedDB.

At first, I chose the Chrome.storage API because it offered a simple and convenient key-value storage system tailored for Chrome extensions. It was easy

to implement and required minimal permissions in the extension's manifest file. One of its key features is the ability to sync data across different instances of Chrome where the user is logged in, though I decided not to use this feature in order to prioritise user privacy. For most of WebGraph's data—bookmarks, tags, and user settings—this API was more than adequate. It could quickly store and retrieve JSON-structured data, which fit well with the extension's graph-based data structure.

However, as the project grew and new features were added, particularly the ability to store snapshots of websites, the Chrome.storage API's limitations became evident. Specifically, it has a 5MB storage limit. This limit quickly became a problem when trying to store base64-encoded image snapshots of websites. For users with a significant number of bookmarks, these snapshots could easily exceed the available storage.

To solve this issue, I decided to adopt a hybrid approach. I continued using the Chrome.storage API for all the textual and structural data—like bookmarks, tags, and settings—because it was still well-suited for these types of data. For the larger data items, such as the website snapshots, I switched to IndexedDB.

```

1 {
2   "spaces": [
3     {
4       "id": "space1",
5       "name": "Example Bookmarks",
6       "sites": [
7         {
8           "dateCreated": "2023-05-15",
9           "favicon": "https://www.google.com/s2/favicons?domain=walkhighlands.co.uk",
10          "id": "site1",
11          "notes": "Great resource for planning hikes",
12          "tags": [
13            "Camping Trip"
14          ],
15          "title": "Hiking Trails in the Highlands",
16          "url": "https://www.walkhighlands.co.uk/",
17          "visits": 12
18        },
19        { ... },
20      ],
21      "tags": [
22        {
23          "id": "tag1",
24          "name": "Camping Trip"
25        },
26        { ... }
27      ]
28    }
29  ]
30 }

```

```

1 [
2   {
3     "key": "site148327",
4     "value": {
5       "id": "site148327",
6       "snapshot": "data:image/jpeg;base64,/9j/4AAQSkZJRg..."
7     }
8   },
9   {
10    "key": "site244120",
11    "value": {
12      "id": "site244120",
13      "snapshot": "data:image/jpeg;base64,/9j/4AAQSkZJRg..."
14    }
15  },
16  {
17    "key": "site840957",
18    "value": {
19      "id": "site840957",
20      "snapshot": "data:image/jpeg;base64,/9j/4AAQSkZJRg..."
21    }
22  }
23 ]

```

Figure 3.4: JSON files of node metadata. Base64 images stored in IndexedDB on the right

3.3 Graph Visualisation

In the design of the WebGraph interface, nodes represent either websites or tags, with their colours indicating the type—sites or tags. The nodes for websites include the site's favicon inside them, visually linking the node to the specific site it represents. Background constraints are defined by groups, which are collections of related tags, such as "Travel Plans" or "Flights," that provide context and organisation within the graph. Groups can be dragged around to position them however the user likes.

3.3.1 D3.js and WebCoLa

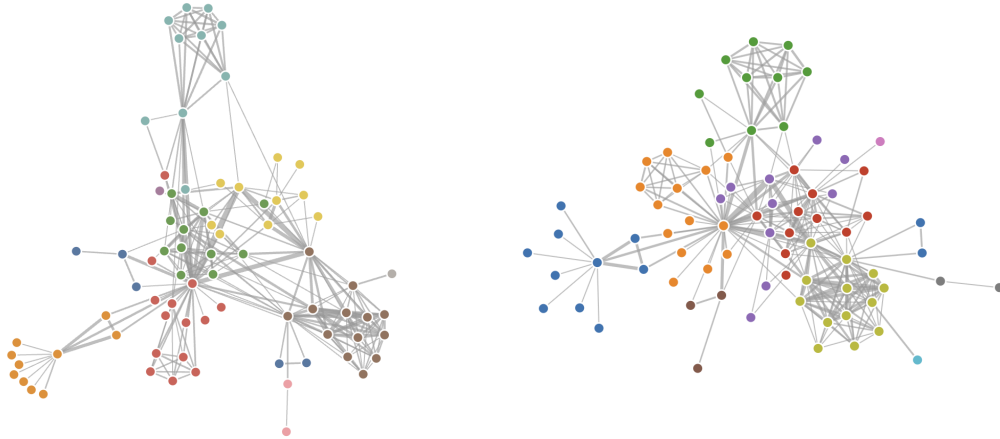


Figure 3.5: Identical data sets represented by vanilla D3.js on the left and WebCoLa on the right. WebCoLa makes use of constraint-based optimisation.

I researched many frameworks for the rendering of the graph, including Cytoscape.js, Sigma.js, Graphology, Pixi.js, and others. The two that I was most drawn to was D3.js and WebCoLa. D3.js is a powerful and versatile data visualisation toolkit with a large community and extensive documentation. It uses the module ‘d3-force’ to implement a velocity Verlet numerical integrator for simulating physical forces on particles, which can be used to visualise networks and hierarchies. WebCoLa is a constraint-based layout library written over the top of D3.js that allows for more customisable grouping of nodes [12].

WebCoLa was my first choice. It would have been a great fit for my app because it’s excellent at drawing various groups and constraints within the renderer. Unfortunately, a lack of documentation meant I could not do what I set out to do in the time frame. One problem I encountered was with overlapping groups. I wanted to create visual clusters of nodes, but I couldn’t get WebCoLa to handle this in a way that looked good and performed well. The groups would often overlap in ways that made the graph difficult to read and interact with.

My decision was to use D3.js alone. While it meant I had to give up customisable groups, the documentation was extensive, and it gave me the control and predictability I needed. Additionally, I knew that it would integrate seamlessly with Chrome. I was able to create my own simplified grouping system using D3’s force simulation, which, while not as sophisticated as WebCoLa’s constraint-based system, was more than adequate for my needs and easier for me to customise and debug.


```

drawGroups(groups) {
  this.groupElements = this.container.selectAll('.group')
  .data(groups)
  .join(
    enter => enter.insert('path', ':first-child')
    .attr('class', 'group')
    .style("fill", (d, i) => this.color(i))
    .style("stroke", (d, i) => d3.rgb(this.color(i)).darker())
    .style("opacity", 0.3),
    update => update,
    exit => exit.remove()
  )
  .attr('d', this.groupPath.bind(this));
}

groupPath(d) {
  if (!d || !d.nodes || d.nodes.length < 2) return "";
  const hull = d3.polygonHull(d.nodes.map(n => [n.x || 0, n.y || 0]));
  return hull ? `M${hull.join("L")}Z` : "";
}

```

Figure 3.6: The functions responsible for drawing groups around tags and their children sites

The functions `drawGroups` and `groupPath` are responsible for visualising groups by drawing and updating SVG path elements within the container. It binds group data to existing elements, adding new ones for incoming data, updating existing ones, and removing those no longer needed. Each group is represented by a convex hull, a polygon that encloses all its nodes, if there are at least two nodes. The `drawGroups` method sets the visual properties of these polygons, such as fill colour, stroke, and opacity, creating a visual representation of grouped data points on the canvas.

3.4 UML and Sequence Diagram

Refactoring

When I first started developing WebGraph, I made the mistake of putting all functionality into a single class, `GraphVisualization`. This class handled rendering, data management, user interactions, settings, and sidebar management. As the project grew, this monolithic approach became unwieldy and difficult to maintain and extend. I realised I needed to refactor the system to improve its modularity and maintainability. The current design, as shown in the UML class diagram, splits the functionality into several specialised classes:

GraphVisualization: This remains the main class, but now it orchestrates the other components rather than implementing all the functionality itself.

GraphRenderer: Handles all aspects of rendering the graph using D3.js.

InteractionHandler: Manages user interactions with the graph and UI elements.

SettingsManager: Deals with loading, saving, and updating graph settings.

SidebarManager: Manages the sidebar content and interactions.

DataManager: Responsible for loading and managing bookmark data.

DataUtils: A utility class with static methods for creating links and groups.

This modular design allows each class to focus on its specific responsibilities, making the code easier to understand, maintain, and extend. It also makes it easier to test individual components in isolation.

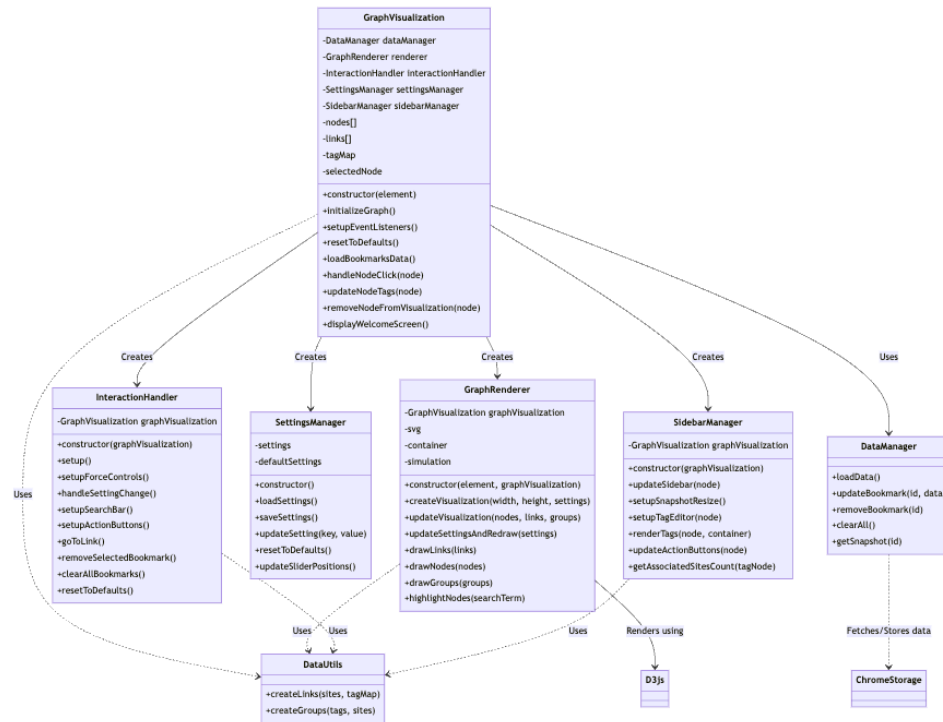


Figure 3.7: UML Diagram

In hindsight, I've learnt valuable lessons about system design. In the future, should plan the structure of methods and classes more carefully from the start, rather than adding everything to one class just because it's convenient in the moment. This experience has taught me the importance of the Single Responsibility Principle [13] and how it contributes to creating more maintainable and scalable software.

This refactoring process, while time-consuming, has resulted in a much more robust and flexible system. It's now easier to add new features or modify existing ones without affecting the entire system.

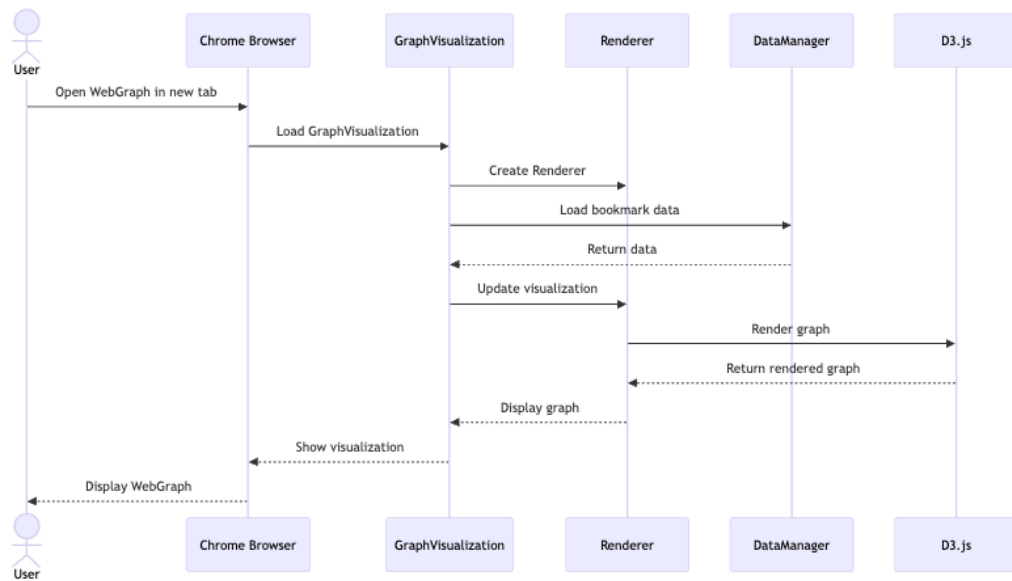


Figure 3.8: Sequence Diagram

The sequence diagram illustrates how the components interact to render the graph. When a user opens WebGraph, the GraphVisualization initialises the other components, loads data through the DataManager, and then uses the GraphRenderer to create and update the visualisation.

The data from the DataManager class, comprising nodes and links, forms the backbone of the graph structure. Once loaded, this information is passed to the GraphRenderer for visualisation. The rendering process begins with the creation of a force-directed simulation using D3.js. This simulation applies various forces to the nodes and links, creating a dynamic, self-organising layout. Forces such as charge (repulsion between nodes), link distance, and centring work together to distribute the elements across the canvas in an aesthetically pleasing and semantically meaningful arrangement.

When changes occur in the underlying data - such as adding or removing bookmarks, modifying tags, or changing the graph settings - the GraphVisualization class triggers an update. This update goes to the GraphRenderer, which then calls the updateVisualization method to reflect these changes immediately.

3.5 Deployment to Chrome Store

Single purpose

An extension must have a single purpose that is narrow and easy to understand. [Learn more](#)

Single purpose description*

To provide users with a visual, interactive graph-based interface for managing and exploring their web bookmarks and associated tags.

133 / 1000

Permission justification

A [permission](#) is either one of a list of known strings, such as 'activeTab', or a [match pattern](#) giving access to one or more hosts.

Remove any permission that is not needed to fulfil the single purpose of your extension. Requesting an unnecessary permission will result in this version being rejected.

storage justification*

The storage permission is necessary for WebGraph to save and retrieve your bookmarks and graph settings locally on your device. This ensures your data persists between browser sessions and allows you to build your personalized web of bookmarks over time.

254 / 1000

activeTab justification*

The activeTab permission is required to capture information about the current tab when you choose to save it as a bookmark. This includes the page title, URL, and a snapshot of the page. The extension only accesses tab information when you explicitly interact with it, ensuring your browsing privacy.

300 / 1000

Figure 3.9: Ensuring I comply with the Chrome Web Store Developer Program Policies

There are several reasons I decided to develop and deploy my application for Google Chrome. It has a vast user base, straightforward deployment and maintenance, and has several developer tools built in.

The deployment of the WebGraph extension to the Google Chrome Web Store was an eye-opening process, revealing both the advantages and challenges inherent in publishing a Chrome extension. While Google's documentation for developers is comprehensive, providing clear guidance on creating and submitting extensions, the actual review process proved to be more opaque and unpredictable than anticipated. Despite following the official guidelines closely, the timeline for the review process was inconsistent, with reports from other developers indicating that approval could take anywhere from a few days to several weeks [14].

One of the primary hurdles encountered during this phase was related to the use of host permissions, which triggered frequent warnings from the extension deployment staging page. The warnings consistently cited the need for an "in-depth review" due to the host permissions requested by the extension, which would have significantly delayed the publishing timeline. This issue is well-documented within the developer community, with other developers expressing similar frustrations on forums like Stack Overflow.

To address these delays and improve the likelihood of passing the review process, several modifications were made to the extension's codebase, particularly in the Chrome extension manifest.json file. In depth look at my permissions was undertaken, and I refined and reduced the scope of permissions. In this process, I lost some minor features, like the ability to use the context menu to open the visualisation.

Additionally, steps were taken to address the "Enhanced Safe Browsing" warning that appears during the app installation process, which flagged the extension as "not trusted." This warning, which new extension developers frequently encounter, indicates that the extension may require additional time for verification to ensure compliance with Google's security standards [19]. It often takes a few months for the developer to become trusted by Google, but again, this process is opaque.

By proactively modifying the extension's code to minimise the use of sensitive permissions and aligning the implementation with Manifest V3 requirements, the extension ultimately navigated the review process more efficiently and it took around three days for my extension to be published. Since publication, updates have usually taken less than a day to be pushed to the deployed app. Despite the initial setbacks, these efforts resulted in the successful publication of WebGraph on the Chrome Web Store, where it is now accessible to users.

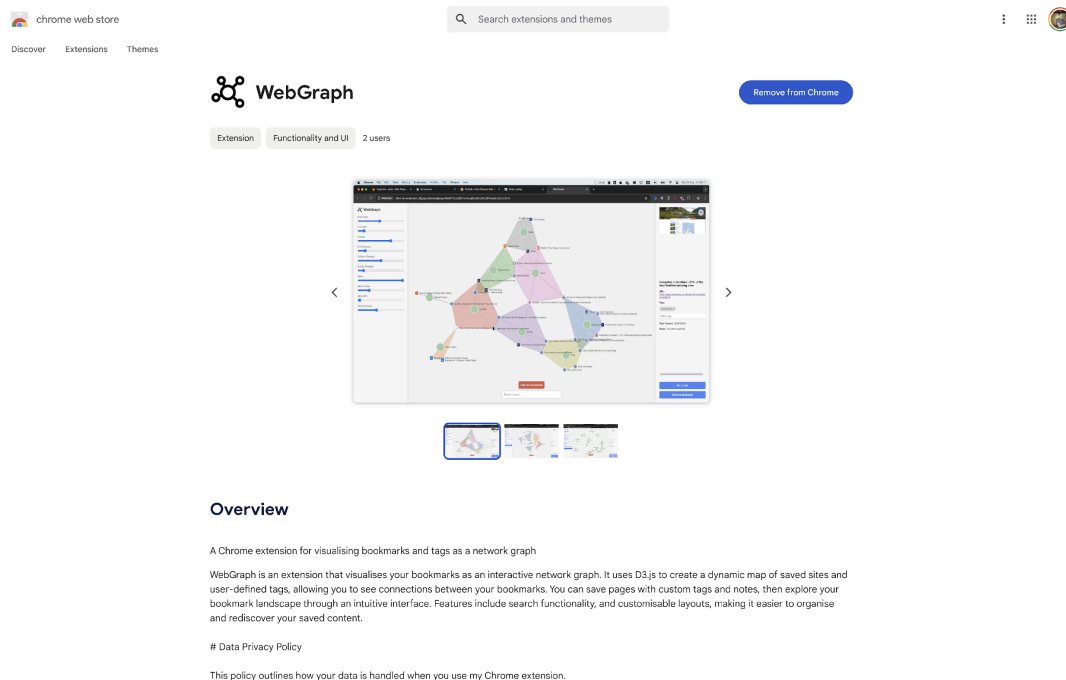


Figure 4: WebGraph's listing on the Web Store

Chapter 4 Testing and Evaluation

The testing and evaluation process was crucial in identifying and resolving issues, ensuring the extension's functionality across different environments, and gathering valuable user feedback.

I began with an online survey with 8 questions with a combination of linear scale and free text answers. The aim was to gather quantitative and qualitative user experience data and to see where the strengths and weaknesses of the app were. It was also a way to quickly find remaining bugs and fix the most crucial UX design choices that were hindering a good user experience.

After the results from my online survey came through, it was clear that there was room for more thorough testing. Since I could not spectate online participants while they completed an online Google Form, I was not collecting a piece of crucial data – the participant's final bookmark data set after completing the tasks. This would be valuable to me because it would allow me to see how the graph visualisation renders data from users other than myself.

Before I carried out user evaluations, I completed a series of tests to make sure that the program was ready for the participants, who kindly gave up their time for this purpose.

4.1 Performance and Optimisation

A comprehensive testing strategy was implemented to ensure the robustness and reliability of WebGraph. This strategy encompassed automated and manual testing methods.

Chrome's built-in performance insights tool was instrumental in analysing and improving WebGraph's performance. This tool provided detailed information about rendering times, CPU usage, and memory consumption, which was particularly crucial for an application rendering potentially hundreds of SVG elements at a high frame rate.

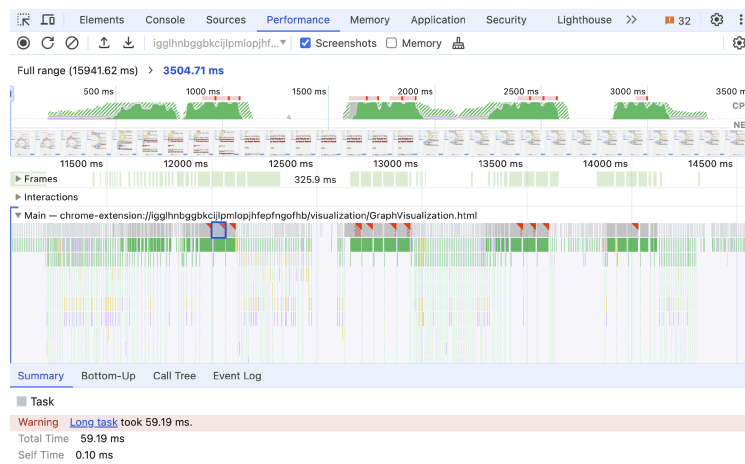


Figure 4.1: Chrome’s built-in performance tools showing high CPU usage for certain tasks

What the performance tests showed was that:

1. Initial render times were very good thanks to d3.js highly optimised rendering engine, and the render times did not noticeably increase unless the number of bookmarks grew beyond expended use.
2. Memory usage grew slowly and steadily over time, indicating potential memory leaks.
3. Zooming into highlighted text after using the search bar caused stuttering. This could be because the highlight nodes method is adding more vector graphics to the renderer. A fix would be to use non-intensive CSS to highlight instead, like simply changing the colour.

More testing was done on different operating systems – both Windows and MacOS – and no differences in functionality or performance were detected. I also wanted to test the application on different hardware tiers. Development and testing were carried out on a M1 Apple Silicon chip, but the Chrome performance tool allows for artificial throttling of the CPU to simulate the program running on lower end hardware. Thus, I was able to ensure that the program ran well on lower end systems as well. There was a noticeable framerate drop when throttling the CPU, but not enough to truly hinder the user experience.

If needed, a radical approach to improve optimisation in the future would be to switch data visualisation entirely to something like Chart.js, a more modern open-source library that focuses on HTML5 Canvas. Another approach would be to use WebGL [15], which might offer better performance for larger graphs. If I were to include a web history functionality in the app in the future, the switch to WebGL might be necessary due to the much larger data set.

Manual Testing

To ensure the functionality and reliability of WebGraph, a comprehensive manual testing script was developed and executed. This script was expanded as new features were added to the extension. The following table outlines the key test cases:

No.	Test
1	Install WebGraph from Chrome Web Store
2	Pin the extension to Chrome toolbar
3	Open WebGraph and verify initial graph state
4	Add a new bookmark and verify it appears in the graph
5	Create a new tag and assign it to a bookmark
6	Search for a specific bookmark or tag
7	Adjust graph visualization parameters using sliders
8	Zoom in/out and pan across the graph
9	Delete a bookmark and verify it's removed from the graph
10	Edit an existing tag
13	Verify correct rendering of website favicons
14	Test graph performance with many bookmarks (>100)

15	Verify correct behaviour when adding duplicate bookmarks/tags
16	Check sidebar information updates correctly when selecting nodes

Table 4.2: Manual testing script

4.2 Online User Evaluation

After testing WebGraph, I thought it would be valuable to evaluate the effectiveness of the application according to the stated objectives. I created a set of tasks and questions using Google Forms and sent them to a mix of computer science university students and some admin staff. Overall, eight participants completed the tasks and the questionnaire, with 6 giving qualitative feedback in the free text box. I was verbally told by some that they were unable to complete the evaluation as they did not use Chrome, which raised an important point. Chrome has the biggest market share of 65.41% [16], but the next most popular browser – Safari is incompatible with WebGraph. The extension should, however work on chromium-based browsers like Brave, Edge, and Arc. I had one individual complete the online survey using Arc browser, and they didn’t experience anything unusual from using Arc.

Participants were provided with a link to the Web Store and asked to complete a series of tasks, including:

1. Search for WebGraph in the Chrome Web Store
2. Pin the extension in your browser
3. Clear all bookmarks
4. Create bookmarks and assign tags to them
5. Adjust graph visualisation parameters using the sliders
6. Search for specific bookmarks or tags
7. Delete a bookmark
8. Delete a tag using the tag editor and replace it

After completing these tasks, the participants were provided with a questionnaire. The main body of questions asked the user to rate aspects of the application on a range of 1-5. These questions and average ratings are provided in table 5.1

Question	Average score (0-5)
How intuitive was navigating the application?	4.25
How aesthetically pleasing was the application?	4.5
How easy was it to organise bookmarks?	3.9
How clear was the graph visualisation?	4.6
How useful were the customisation options?	4
How much do you think this application can improve bookmark organisation?	4.6
How likely would you recommend this application to someone looking to better manage their bookmarks?	4.6

Table 4.3: Questions and Average Ratings

Overall, the extension was received positively with an average score of 4.34 from its 8 participants. They claimed they found the application intuitive, aesthetically pleasing, and they found the visualisation to be clear. In the additional comments section, participants were allowed to provide more detailed feedback. Some key points from the feedback of the online and in person evaluations include the following:

- A user experienced issues with deleting tags, which were subsequently addressed and fixed. A bug was allowing two separate tags to share the same primary key.
- A user was confused about how to add bookmarks, so I added a simple welcome screen on fresh installs with some brief instructions.
- Users requested a setting to adjust text size, so I implemented it as it was a quick addition.
- Another user said that they didn't like the way different groups were "flying away from each other". I had gotten familiar with this phenomenon and didn't realise it was so undesirable. My solution was to add a centring force like gravity using d3-force. In the sidebar it's labelled as "Gravity Strength".
- During the in-person evaluation, I noticed that one participant wasn't instinctively refreshing the visualisation webpage when the JSON was updated by the DataManager class. I was so familiar with refreshing the page that I couldn't imagine anyone not thinking to do it. I added an additional button called "refresh" that should prompt users to refresh the page after data modification. It functions the same way as pressing Ctrl + R.

I believe that the lowest score for question 3 "How easy was it to organise bookmarks?" was due to users struggling to keep their groups centred in the viewport. I expect that this value would increase now that an artificial gravity force has been added. Overall, the user evaluation provided valuable insights that led to significant improvements in WebGraph's usability and functionality.

Google Reviews

At the time of writing, WebGraph has received 2 reviews on the Web Store averaging a score of 4.5 out of 5. Accompanying review text describes the app as "fun", "easy to use", and "very useful". Although the current Google reviews are a very small set of results, it is worth mentioning because it's a passive form of constant evaluation. It is a type of user evaluation that does not require any input from the developer. It does not replace user evaluations, but it is an unintended bonus of deploying an app on the Web Store.

4.3 In Person Evaluation



Figure 4.4: Two in-Person user evaluation bookmarks after 10 minutes. User A is on the left and User B is on the right.

Network graphs are more useful when you can compare them to other network graphs [17]. I thought it was necessary to gather a small number of participants to complete an in-person user evaluation. There were three participants in total, Users A, B, and C. They were given a brief demonstration of the app, and then given 10 minutes to populate their own data set of bookmarks and tags. They were purposefully not given too much information on how they should organise their bookmarks because I wanted to spectate how they would use the app without preconceptions.

Comparing the participants graphs after 10 minutes of use provides a detailed representation of their web browsing patterns, revealing distinct clusters of activity around specific categories or topics. The presence of these clusters suggests that users organise their bookmarks in a manner that reflects their primary interests or tasks. For example, one user's graph prominently features clusters associated with shopping, social media, and administrative tasks, while another user's graph highlights clusters related to media consumption, financial management, and reading. The proximity and connections between clusters also implies that users perceive certain activities as related or complementary, as evidenced by the overlap between social media and messaging applications.

All three users modified the graph settings through the sidebar, to adjust for their tastes. This level of interaction indicates that users are tailoring the tool to suit their specific needs, enhancing the utility of the visualisation in managing and organising their online activities. Additionally, the use of tags as single, double or triple tags reflects a conscious effort by the user to categorise and retrieve information more efficiently. This feature appears to be an integral part of the user experience.

Chapter 5 Conclusion

5.1 Achievements

Reflecting on the project requirements and user stories, I managed to achieve a hit rate of 100% (7/7) for Must Haves, 67% (6/9) for Should Haves, and 25% (1/4) for Could Haves. User testing has shown that the application is successful in representing semantic relationships between websites and user-defined categories/tags. It also showed that WebGraph improved bookmark organisation and information retrieval, allowing users to find their saved content more easily. And finally, testing proved that the application was performant across a variety of operating systems, hardware, and browsers. It's a personal achievement that I was able to integrate the extension seamlessly with the browser, which was a central objective of the project.

5.2 Limitations

I started this project with the intention of exploring three main types of web browsing data: tabs, bookmarks, and history. My vision was to develop an app that could encompass all three data types and create custom visualisations for each. However, given the scope of the project and the limited time available, I had to narrow my focus to just bookmark visualisation. Although this was a reasonable decision, I still consider it a limitation because it meant compromising on my original vision.

A significant portion of my time was spent attempting to get various bundlers, frameworks, and libraries to work, specifically Webpack, React, and Vite. Unfortunately, this time could have been better spent improving the fundamentals of the app and adding user stories. My decision to use these tools was based on the assumption that they would enhance the app in the long term, making it easier to maintain and deploy. At the small cost of adding some boilerplate at the beginning of the project, I thought this was worth the trade-off. I was wrong and think I should have adhered to the design principle “Keep It Simple, Stupid” [18].

I ended up wasting about a week trying to integrate these technologies with my prototype graphs. It became a classic case of having a solution in search of a problem. The app was simple enough that HTML and CSS were sufficient for the Minimum Viable Product (MVP). React was not necessary, and Webpack's bundling capabilities were not worth the hassle of learning a new technology within such a short time frame.

This experience taught me an important lesson: do not adopt a solution simply because you think it will add polish to your software. Instead, focus on solving the actual problems at hand.

Another limitation I encountered was my inability to use the constraint-based graph layout powered by WebCoLa. I had hoped to implement WebCoLa and still wish to do so, but the lack of documentation made it nearly impossible to achieve the custom overlapping group paths I needed.

5.3 Future Work

Looking ahead, there are several ways of enhancing WebGraph's functionality and user experience. Some were missed initial requirements, and some became clear to me as I was developing the app.

One significant development would be the creation of an algorithm to convert Zotero web references into a WebGraph JSON format. Since Zotero's database similarly uses the attributes URL, notes, and tags, this would not be too difficult to accomplish. This integration would allow academic users to seamlessly transition their existing research databases into the WebGraph ecosystem, potentially expanding the tool's utility in academic contexts and allow these users to see more semantic meaning in their web references. On the topic of Zotero, it uses a different method to capture snapshots of a website. It captures the basic HTML of a website instead of an image. This may be superior because it preserves things like hyperlinks and does not need image compression, which can cause ugly blurriness. This is a relatively straightforward change that I plan to push to WebGraph in a future update.

Another direction is the implementation of a tree-based history graph. This feature would provide users with a more comprehensive view of their browsing patterns over time, allowing for deeper insights into their web usage habits and allowing them to easily go back in time in their web history, navigate, and find past content more easily.

Incorporating a to-do list feature could broaden WebGraph's appeal. Many users currently employ their browser tabs as makeshift task lists; by integrating this functionality directly into WebGraph, we could offer a more structured and visually engaging way to manage tasks. This could be further enhanced with gamification elements, such as the satisfying action of "popping bubbles" to complete tasks, adding an element of fun to productivity.

A simple visual enhancement that was considered in the requirements stage was dynamically adjust the size of site nodes based on visit frequency or other metrics. This would add an extra layer of encoded data into the visualisation, offering users a top-down understanding of their most frequently accessed resources.

These future developments would expand WebGraph's functionality and create a more intuitive, engaging, and productive web browsing experience for a variety of users.

Chapter 6

Chapter 7 References

- [1] Ramis, A. J. (2023, October 24). Stop, Before You Close This Tab (or Any Others) The New York Times. <https://www.nytimes.com/2023/10/24/magazine/browser-tabs-forever.html>
- [2] stefanXO/Tab-Manager-Plus. (2024). [JavaScript]. stefanXO Labs. <https://github.com/stefanXO/Tab-Manager-Plus> (Original work published 2018)
- [3] 00—Start here—Obsidian Hub—Obsidian Publish. (n.d.). Obsidian Hub. Retrieved 30 August 2024, from <https://publish.obsidian.md/hub/00+-+Start+here>
- [4] Tweedie, S. (n.d.). This Mesmerizing Map Of The Internet Lets You Explore The Web's Tiny Connections. Business Insider. Retrieved 30 August 2024, from <https://www.businessinsider.com/what-does-a-map-of-the-internet-look-like-2014-10>
- [5] Mason, T. (2019, May 9). Graph File Systems. A File Systems Geek. <https://fsgeek.ca/2019/05/09/graph-file-systems/>
- [6] Barnard, G. A., & Fein, L. (1958). An information filing and retrieval system for the engineering and management records of a large-scale computer development project. *American Documentation*, 9(3), 208–213. <https://doi.org/10.1002/asi.5090090307>
- [7] Shneiderman, B. (2003). The Eyes Have It: A Task by Data Type Taxonomy for Information Visualizations. In B. B. Bederson & B. Shneiderman (Eds.), *The Craft of Information Visualization* (pp. 364–371). Morgan Kaufmann. <https://doi.org/10.1016/B978-155860915-0/50046-9>
- [8] ‘D3-Force | D3 by Observable’. Accessed 12 July 2024. <https://d3js.org/d3-force>.
- [9] D3.polygonHull / D3. (n.d.). Observable. Retrieved 30 August 2024, from <https://observablehq.com/@d3/d3-polygonhull>
- [10] Berge, Claude. ‘Graph Theory’. *The American Mathematical Monthly* 71, no. 5 (1964): 471–81. <https://doi.org/10.2307/2312582>.
- [11] Bipartite Graph—An overview | ScienceDirect Topics. (n.d.). Retrieved 30 August 2024, from <https://www.sciencedirect.com/topics/engineering/bipartite-graph>
- [12] ‘Cola.js: Constraint-Based Layout in the Browser’. Accessed 2 July 2024. <https://ialab.it.monash.edu/webcola/index.html>.
- [13] freeCodeCamp.org. ‘SOLID Definition – the SOLID Principles of Object-Oriented Design Explained’, 26 April 2022. <https://www.freecodecamp.org/news/solid-principles-single-responsibility-principle-explained/>.

- [14] Davide. (2020, May 16). Answer to ‘Google Chrome Extension Review Time process’ [Online post]. Stack Overflow. <https://stackoverflow.com/a/61835851>
- [15] yWorks, the diagramming. (2018, March 28). *SVG, Canvas, WebGL? Visualization options for the web*. yWorks, the Diagramming Experts. <https://yworks.com/blog/svg-canvas-webgl>
- [16] *Internet browser market share 2012-2024*. (n.d.). Statista. Retrieved 30 August 2024, from <https://www.statista.com/statistics/268254/market-share-of-internet-browsers-worldwide-since-2009/>
- [17] Visualizing Topic Models with Force-Directed Graphs. (2012, December 3). *The Stone and the Shell*. <https://tedunderwood.com/2012/12/02/visualizing-topic-models-with-force-directed-graphs/>
- [19] *This extension is not trusted by Enhanced Safe Browsing*. (n.d.). Retrieved 30 August 2024, from <https://groups.google.com/a/chromium.org/g/chromium-extensions/c/3g8tu00by2g>
- [18] The Interaction Design Foundation. ‘What Is Keep It Simple, Stupid (KISS)? — Updated 2024’. Accessed 29 August 2024. <https://www.interaction-design.org/literature/topics/keep-it-simple-stupid>.

Appendix A Ethics Checklist

**School of Computing Science
University of Glasgow**

Ethics checklist form for 3rd/4th/5th year, and taught MSc projects

This form is only applicable for projects that use other people ('participants') for the collection of information, typically in getting comments about a system or a system design, getting information about how a system could be used, or evaluating a working system.

If no other people have been involved in the collection of information, then you do not need to complete this form.

If your evaluation does not comply with any one or more of the points below, please contact the Chair of the School of Computing Science Ethics Committee (matthew.chalmers@glasgow.ac.uk) for advice.

If your evaluation does comply with all the points below, please sign this form and submit it with your project.

-
1. Participants were not exposed to any risks greater than those encountered in their normal working life.

Investigators have a responsibility to protect participants from physical and mental harm during the investigation. The risk of harm must be no greater than in ordinary life. Areas of potential risk that require ethical approval include, but are not limited to, investigations that occur outside usual laboratory areas, or that require participant mobility (e.g. walking, running, use of public transport), unusual or repetitive activity or movement, that use sensory deprivation (e.g. ear plugs or blindfolds), bright or flashing lights, loud or disorienting noises, smell, taste, vibration, or force feedback

2. The experimental materials were paper-based, or comprised software running on standard hardware. *Participants should not be exposed to any risks associated with the use of non-standard equipment: anything other than pen-and-paper, standard PCs, laptops, iPads, mobile phones and common hand-held devices is considered non-standard.*

3. All participants explicitly stated that they agreed to take part, and that their data could be used in the project.

If the results of the evaluation are likely to be used beyond the term of the project (for example, the software is to be deployed, or the data is to be published), then signed consent is necessary. A separate consent form should be signed by each participant.

Otherwise, verbal consent is sufficient, and should be explicitly requested in the introductory script.

4. No incentives were offered to the participants.

The payment of participants must not be used to induce them to risk harm beyond that which they risk without payment in their normal lifestyle.

5. No information about the evaluation or materials was intentionally withheld from the participants.
Withholding information or misleading participants is unacceptable if participants are likely to object or show unease when debriefed.
6. No participant was under the age of 16.
Parental consent is required for participants under the age of 16.
7. No participant has an impairment that may limit their understanding or communication.
Additional consent is required for participants with impairments.
8. Neither I nor my supervisor is in a position of authority or influence over any of the participants.
A position of authority or influence over any participant must not be allowed to pressurise participants to take part in, or remain in, any experiment.
9. All participants were informed that they could withdraw at any time.
All participants have the right to withdraw at any time during the investigation. They should be told this in the introductory script.
10. All participants have been informed of my contact details.
All participants must be able to contact the investigator after the investigation. They should be given the details of both student and module co-ordinator or supervisor as part of the debriefing.
11. The evaluation was discussed with all the participants at the end of the session, and all participants had the opportunity to ask questions.
The student must provide the participants with sufficient information in the debriefing to enable them to understand the nature of the investigation. In cases where remote participants may withdraw from the experiment early and it is not possible to debrief them, the fact that doing so will result in their not being debriefed should be mentioned in the introductory text.
12. All the data collected from the participants is stored in an anonymous form.
All participant data (hard-copy and soft-copy) should be stored securely, and in anonymous form.

Project title WebGraph: A Bookmark Management and Visualization Plugin for Google Chrome

Student's Name Raphael Taylor

Student Number 2940447T

Student's Signature Raphael Taylor

Supervisor's Signature Yitian Tang

Date 28/08/24

Ethics checklist for projects